

Open-LLM-VTuber on NVIDIA DGX Spark

Complete installation and operation guide for running Open-LLM-VTuber on a NVIDIA DGX Spark (GB10 Grace-Blackwell, 128 GB unified memory). All AI models run fully locally on the same GPU simultaneously.

Architecture

Four AI services run concurrently: llama-server (llama.cpp, CUDA) serving the HauhauCS Qwen3.6-35B-A3B uncensored LLM on port 8080, faster-whisper large-v3-turbo (CUDA) for speech recognition, MeloTTS or Qwen3-TTS for speech synthesis, and Silero VAD (CPU) for voice activity detection. The Open-LLM-VTuber FastAPI server binds port 12393.

Unified Memory Budget

Component	Memory
LLM — Qwen3.6-35B-A3B Q5_K_P GGUF	~28 GB
ASR — faster-whisper large-v3-turbo	~3 GB
TTS — MeloTTS (or Qwen3-TTS 0.6B)	~2 GB (~1 GB)
VAD — Silero VAD	~0.1 GB
KV cache — 128K context at q8_0	~8 GB
OS overhead	~8 GB
TOTAL	~49 GB (79 GB free of 128 GB)

Model Background — HauhauCS Uncensored

HauhauCS (<https://huggingface.co/HauhauCS>) creates lossless uncensored models. Their Qwen3.6 releases score 0/465 refusals — completely unlocked, never refuses a prompt. Aggressive variant strips reasoning for raw answers; Balanced variant keeps reasoning inline for agentic stability.

Qwen3.6-35B-A3B is a Mixture-of-Experts with 35B total / ~3B active per token. K_P (Perfect) quants preserve quality 1-2 levels above base at ~5-15% larger size. Q5_K_P at ~28 GB is the sweet spot on 128 GB unified memory.

Quick Install

```
git clone https://github.com/subject2change/Open-LLM-VTuber.git && cd Open-LLM-VTuber && bash examples/dgx-spark/setup.sh
```

For Qwen3-TTS: `bash examples/dgx-spark/setup.sh --with-qwen3-tts`

Manual Install — Step by Step

1. System Dependencies

```
sudo apt-get update && sudo apt-get install -y build-essential cmake curl git make libopenblas-dev libomp-dev python3-dev
```

2. Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

3. Python Dependencies

```
cd Open-LLM-VTuber && uv sync
```

```
uv pip install faster-whisper melo-tts==0.1.2 pytest
```

4. Build llama.cpp from Source

```
git clone --depth 1 https://github.com/ggml-org/llama.cpp build/llama.cpp
```

```
cd build/llama.cpp && cmake -B build -DGGML_CUDA=ON . && cmake --build build -j$(nproc) --target llama-server
```

```
sudo cp build/bin/llama-server /usr/local/bin/
```

5. Download LLM Model

Download Qwen3.6-35B-A3B Aggressive Q5_K_P (~28 GB) from HauhauCS on Hugging Face.

6. Deploy Config & Hermes Manifest

```
cp examples/dgx-spark/conf.example.yaml conf.yaml
```

Then create .hermes/environment.json with start=uv run run_server.py on port 12393.

Running the VTuber

```
Terminal 1: llama-server -m models/filename.gguf --host 127.0.0.1 --port 8080 -ngl 99 -c 131072 --jinja --mlock -fa on --cache-type-k q8_0 --cache-type-v q8_0 --chat-template-kwarg 'enable_thinking: false'
```

```
Terminal 2: cd Open-LLM-VTuber && uv run run_server.py --verbose
```

If using Qwen3-TTS, start a third terminal: qwen3-tts serve --model Qwen/Qwen3-TTS-12Hz-0.6B-Base --quant q4

Then open <http://localhost:12393>.

First-Boot Downloads

On first run, ASR/TTS backends auto-download: faster-whisper large-v3-turbo (~3 GB) into models/whisper/, MeloTTS EN-Default (~0.5 GB). These can take minutes. Subsequent boots are instant. setup.sh pre-caches faster-whisper when possible.

Verification — hermes verify

hermes verify --json --ready-timeout 120 — runs bootstrap, tests, starts server, polls readiness, tears down.

Model Selection Guide

LLM Options — All Uncensored HauhauCS

Model	Quant	Size	Notes
Qwen3.6-35B-A3B Aggressive (default)	Q5_K_P	~28 GB	Best quality/latency for 128 GB
Qwen3.6-27B Balanced	Q6_K_P	~23 GB	Dense 27B; stable for agentic chains
Qwen3.6-35B-A3B Aggressive	Q4_K_M	~21 GB	Lighter, faster startup
Gemma4-12B QAT Balanced + MTP	Q4_K_M	~7.4 GB	60% faster via speculative decoding
Qwen3.6-35B-A3B Aggressive	Q8_K_P	~44 GB	Maximum quality

TTS Options

Model	GPU	Quality	Notes
MeloTTS (default)	~2 GB	4/5	Fully local, GPU-accelerated
Qwen3-TTS (recommended)	~1-3 GB	5/5	Studio-quality, standalone server; see dedicated section
edge-tts	0 GB	5/5	Zero GPU, needs internet
Piper TTS	0 GB (CPU)	3/5	CPU-only, many voices
Sherpa-ONNX TTS	~1 GB	3/5	GPU-accelerated

ASR Options

Model	VRAM	Latency	Accuracy
faster-whisper large-v3-turbo (default)	~3 GB	~100ms	5/5
faster-whisper medium	~1.5 GB	~50ms	4/5
sherpa-onnx sense-voice	~1 GB (CPU)	~50ms	4/5
whisper.cpp small	~1 GB	~80ms	3/5

Qwen3-TTS — Studio-Quality Local TTS (Recommended)

Qwen3-TTS is Alibaba's state-of-the-art TTS model: 10 languages, voice cloning from reference audio (CustomVoice), and text-based voice design (VoiceDesign). Quality rivals ElevenLabs. Runs as standalone server alongside llama-server, exposing OpenAI-compatible `/v1/audio/speech` on port 8765.

One-line install: `curl -fsSL`

```
https://github.com/darkautism/qwen3-tts/raw/refs/heads/master/deploy/systemd/install-frontend.sh |  
bash
```

This installs qwen3-tts as a systemd user service, downloads the 1.7B Base model (~1.5 GB), starts API on port 8765. Then set `tts_model: openai_tts` in `conf.yaml` pointing to `http://127.0.0.1:8765/v1`.

Manual: `git clone https://github.com/darkautism/qwen3-tts.git && cd qwen3-tts && cargo build --release`. Run: `./target/release/qwen3-tts serve --model Qwen/Qwen3-TTS-12Hz-0.6B-Base --quant q4` (fast, ~1 GB VRAM) or with 1.7B-Base `--quant q5` (best quality, ~3 GB VRAM).

Qwen3-TTS Variants

Variant	Params	Download	VRAM	Best For
0.6B Base	0.9B	~0.8 GB	~1 GB	Fastest, lowest VRAM
1.7B Base	2B	~1.5 GB	~3 GB	Voice cloning from reference
1.7B CustomVoice	2B	~1.5 GB	~3 GB	Cloning with custom tuning
1.7B VoiceDesign	2B	~1.5 GB	~3 GB	Describe a voice in words

Tuning for Lower VRAM

For GPUs with less than 32 GB VRAM: use edge-tts (zero GPU), Gemma4-12B (~7.4 GB), faster-whisper medium with int8, and reduce context to -c 4096.

Troubleshooting

CUDA Out of Memory

Use smaller LLM (Gemma4-12B ~7.4 GB or Qwen3.6-35B-A3B Q4_K_M ~21 GB). Switch to edge-tts or Qwen3-TTS 0.6B (both zero/low GPU). Use faster-whisper medium int8. Reduce context to -c 4096.

First-Boot Timeout

ASR downloads ~3 GB on first boot. Server becomes ready after download. Pass --ready-timeout 300 to hermes verify.

hermes verify Detects Wrong Port

Auto-detector assumes uvicorn main:app --port 8000. Install manifest at .hermes/environment.json (step 8 of setup.sh).

Frontend Shows Not Found

git submodule update --init --recursive

Key Design Decisions

Why openai_compatible_llm instead of llama_cpp_llm? The built-in provider loads GGUF in-process with few config options. llama-server as standalone gives full control over GPU offloading, context, flash attention, KV cache, and keeps the model loaded between VTuber restarts.

Why Q5_K_P instead of Q4_K_M? K_P preserves quality 1-2 levels above base at ~5-15% larger size. At ~28 GB there is still ~79 GB free on 128 GB — quality improvement is effectively free.

License

Open-LLM-VTuber: MIT. llama.cpp: MIT. Model files (HauhauCS, Qwen3-TTS): review model cards on Hugging Face before redistribution.

Repository

Fork: <https://github.com/subject2change/Open-LLM-VTuber> — Branch: dgx-spark-setup (PR #1)

Setup files: `examples/dgx-spark/conf.example.yaml`, `examples/dgx-spark/setup.sh`

Model sources: <https://huggingface.co/HauhauCS>,
<https://huggingface.co/collections/Qwen/qwen3-tts>